

第10章 泛型与元编程

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 C 的 `void*` / 宏 / 手写多份代码的痛点,以及泛型要解决的问题
- 理解 JS 的动态类型天然"泛型":一个函数服务所有类型,代价是类型安全
- 了解 TypeScript 泛型:为 JS 动态代码补上静态约束的方向指引
- 掌握 Proxy:13 种拦截操作,理解"代理即元编程"
- 掌握 Reflect:与 Proxy 一一对应的反射工具,以及替代老 Object 方法的场景
- 掌握原型链动态操纵与动态元编程的边界
- 理解各语言"编译期算力"对照:生成代码 vs 擦除 vs 运行时动态

💡 从 C 到 JavaScript:本章主题如何迁移

C 想写"一个函数处理任何类型",只有三条路:`void*` + 回调(类型安全靠自觉,还要传长度)、宏(预处理展开,调试地狱)、复制粘贴 N 份(维护噩梦)。C++ 的模板、Java 的泛型把这条路走成了语言特性;JS 直接不设防:函数参数不检查类型,天然接受一切——这就是动态类型,泛型问题的"另类解法"。

代价是类型安全:传错类型不报错,运行到一半才炸。所以 JS 社区的方向是渐进类型:TypeScript 在"写法上像 JS、类型上像 Java"的轨道上给泛型补齐——`function stack<T>()` 这样的声明在第14章细讲,本章先建立"泛型在 JS 生态里长什么样"的坐标。

JS 的元编程则完全在运行时:Proxy 拦截对象的读写,像给对象装"窃听器 + 门卫";Reflect 提供与拦截一一对应的原始操作。Vue 3 的响应式、MobX、各种 ORM 的魔法底层都是 Proxy。C 的宏在编译期改写代码,JS 的 Proxy 在运行期改写行为——一个是预处理器,一个是运行时钩子,这是两种元编程哲学。

最后记住一张对照表:C/C++ 的模板在编译期生成代码(快但二进制膨胀);Java 的泛型在编译期擦除类型(兼容但反射受限);JS 的"泛型"在运行期动态分发(最灵活、最慢)。三选一,没有免费的午餐。

📦 前置知识自查

- C 的 `void*`、函数指针回调、宏 `#define` 泛化
- C++ 模板的直观认识(可选,不要求)
- 第1章对象与属性、第6章原型链
- 第4章 this 绑定(Proxy 的 get 拦截会用到)

🚀 本章学习路线

1. **第一阶段(泛型坐标)**:读 §10.1–§10.3,理解"动态类型天然泛型"与 TS 的方向
2. **第二阶段(Proxy/Reflect 核心)**:读 §10.4–§10.5,亲手写日志代理与校验代理
3. **第三阶段(边界与对照)**:读 §10.6–§10.7,理解元编程的代价与生产判断
4. **第四阶段(实验)**:完成章末"泛型栈 + 日志代理 + 深代理"
5. **验收标准**:能写出 Proxy 拦截 get/set 的代码,并说明 Reflect 在拦截器里的作用

本章知识导图

- 第10章 泛型与元编程
 - §10.1 为什么需要泛型:C 的 void*/宏痛点
 - §10.2 JS 的动态类型:天然"泛型"与代价
 - §10.3 TypeScript 泛型方向指引
 - §10.4 Proxy:13 种拦截与生产场景
 - §10.5 Reflect:与拦截一一对应
 - §10.6 原型链操纵与动态元编程边界
 - §10.7 编译期算力对照表与生产判断

§10.1 为什么需要泛型:先看 C 的三条路

C 的方案	痛点	JS 的对应
void* + 长度参数	类型安全归零,解引用全靠自觉	参数不检查类型(同样靠自觉,但少了解引用)
宏 #define MAX(a,b)	无类型检查、副作用隐患、调试困难	普通函数即可(JS 函数不挑参数)
复制多份实现	维护噩梦,改一处漏三处	一个函数全类型通用

```
// C 的 void* 栈(示意,伪代码):
// void push(Stack* s, void* item, size_t size); ← 每次都要传 size,取回还要强转
// int* p = (int*)pop(s); ← 强转错了不报错

// JS 天然通用:一个栈函数服务任何类型
function createStack() {
  const items = [];
  return {
    push(item) { items.push(item); }, // 不检查类型,number/string/对象都行
    pop() { return items.pop(); },
    peek() { return items[items.length - 1]; },
    get size() { return items.length; },
  };
}
const intStack = createStack();
intStack.push(42);
const strStack = createStack();
strStack.push('hello');
const mixed = createStack();
mixed.push({ a: 1 }, '任意', 3.14); // 混着放都合法
console.log(intStack.peek(), strStack.peek(), mixed.size); // 42 hello 3
```

⚠ 常见误区 10.1:"泛型 = 参数类型任意"是危险简化

动态类型的"任意"没有上限:传错类型 **不会在调用处报错**,而是在函数内部运行时炸,或更糟——静默产出错误结果 (`stack.push('42')` 后 `pop() * 2` 得 NaN)。生产对策:入口处做运行时校验(第14章的 JSDoc/TS 是系统性解法)。泛型的价值不是"能传任意类型",而是"只允许满足约束的类型"。

§10.2 动态类型的"泛型"与内置泛型结构

```
// 内置"泛型"结构:Map/Set 接受任意键值类型
const map = new Map();
map.set('name', '李雷');
map.set(42, '数字键');           // 键可以是任何类型
map.set({ id: 1 }, '对象键');     // 对象键(引用相等)
console.log(map.get(42), map.get({ id: 1 })); // 数字键 undefined(不同引用!)

// Set:去重任意类型
const set = new Set([1, 2, 2, 3, 3, 3]);
console.log(set.size);           // 3,自动去重

// 用工厂函数模拟"泛型容器":闭包持有类型约束
function createTypedStack(validate) {
  const items = [];
  return {
    push(item) {
      if (!validate(item)) throw new TypeError('类型不匹配');
      items.push(item);
    },
    pop() { return items.pop(); },
  };
}
const onlyNumbers = createTypedStack((v) => typeof v === 'number');
onlyNumbers.push(1);
// onlyNumbers.push('x');           // ✗ TypeError:类型不匹配
console.log(onlyNumbers.pop());    // 1

// 运行时"类型约束"的通用写法:先校验再使用(生产必备模式)
function assertNumber(v, what = '值') {
  if (typeof v !== 'number' || !Number.isFinite(v)) {
    throw new TypeError(`${what} 必须是有限数字,收到:${v}`);
  }
}
function divide(a, b) {
  assertNumber(a, 'a');
  assertNumber(b, 'b');
  return a / b;
}
console.log(divide(10, 4));        // 2.5
```

§10.3 TypeScript 泛型:方向指引

JS 的下一步是 TypeScript(第14章详讲)。TS 在 JS 之上加类型标注, **编译期检查**、**运行期擦除**。泛型是 TS 的核心能力之一,读框架源码(React、Vue、Express 的类型声明)必须认识它。这里给出最小语法,建立"泛型长什么样"的坐标:

```
// TypeScript 泛型(示意,需 .ts 文件运行):
// function first<T>(arr: T[]): T | undefined {
//   return arr[0];
// }
// const a = first<number>([1, 2, 3]); // a: number | undefined
// const b = first(['x']); // 自动推导 T = string

// 泛型约束(extends):
// function len<T extends { length: number }>(x: T): number {
//   return x.length; // T 必须是"有 length 的对象"
// }

// 泛型数据结构:
// class Stack<T> {
//   private items: T[] = [];
//   push(item: T): void { this.items.push(item); }
//   pop(): T | undefined { return this.items.pop(); }
// }
// const s = new Stack<number>();
// s.push(1);
// // s.push('x'); // ❌ 编译期报错,类型安全回到编译期

// 生产坐标:小项目 JSDoc 起步(第14章),中大型项目引入 TS;
// 泛型在框架类型里的出现频率 = "组件 Props 泛型"、"工具类型"如
// Partial<T> / Pick<T, K> / ReturnType<T>,读类型声明时按"参数化类型"理解即可
console.log('TS 泛型为 JS 补上编译期类型安全(详见第14章)');
```

§10.4 Proxy:对象的运行时门卫

Proxy 给对象套一层"拦截器":所有对目标对象的操作(get/set/has/delete 等 13 种)先经过处理器(handler)函数,可以改写、记录、校验、拒绝。这是 JS 元编程的主舞台。

```

// 日志代理:记录每一次属性访问与修改
const target = { name: '李雷', score: 90 };
const logger = new Proxy(target, {
  get(obj, prop) {
    console.log(`[get] ${String(prop)}`);
    return Reflect.get(obj, prop); // 转发原始操作
  },
  set(obj, prop, value) {
    console.log(`[set] ${String(prop)} = ${value}`);
    return Reflect.set(obj, prop, value); // 必须返回布尔(是否成功)
  },
});
logger.score; // [get] score
logger.score = 95; // [set] score = 95
console.log(logger.score); // 95

// 校验代理:写保护 + 类型检查
const account = { balance: 0 };
const safe = new Proxy(account, {
  set(obj, prop, value) {
    if (prop === 'balance') {
      if (typeof value !== 'number') throw new TypeError('余额必须是数字');
      if (value < 0) throw new RangeError('余额不能为负');
    }
    return Reflect.set(obj, prop, value);
  },
  deleteProperty(obj, prop) {
    if (prop === 'balance') return false; // 禁止删除关键字段
    return Reflect.deleteProperty(obj, prop);
  },
});
safe.balance = 100;
// safe.balance = -5; // ✖ RangeError
// delete safe.balance; // false, 删除被拒绝
console.log(safe.balance); // 100

// 隐藏敏感属性:读取时拦截
const secret = { user: 'admin', password: 'hunter2' };
const guarded = new Proxy(secret, {
  get(obj, prop) {
    if (prop === 'password') return '***';
    return Reflect.get(obj, prop);
  },
  has(obj, prop) {
    return prop === 'password' ? false : Reflect.has(obj, prop);
  },
});
console.log(guarded.password); // ***
console.log('password' in guarded); // false ← 假装不存在

```

⚠ 常见误区 10.2:Proxy 不透明,this 与身份陷阱

① 代理不改变原对象:对 proxy 的修改会落到 target 上,但 `proxy === target` 为 false,混用两者会出"改了代理原对象没变"的错觉(其实变了,是引用比较的问题);② 方法内 this:对象方法内部访问 this 走的是 target 而不是 proxy,拦截不到,需用 `this = proxy` 绑定(生产里即 Vue 响应式包裹时处理的问题);③ 13 种操作各返回类型不同,set/delete 必须返回布尔。

§10.5 Reflect:与拦截——对应的反射工具

Reflect 提供与 Proxy 13 种拦截——对应的原始操作。两个用途:① 在拦截器内部"转发原始行为";② 替代部分老 Object 方法(返回值更友好)。

```
// 用途一:拦截器内转发(上一节的 get/set 都在用 Reflect)
// 关键:Reflect.set 返回布尔表示成功,Object.defineProperty 抛异常—Proxy 规范要求布尔

// 用途二:替代老 API
const obj = { a: 1, b: 2 };
console.log(Reflect.has(obj, 'a')); // true,替代 'a' in obj
console.log(Reflect.ownKeys(obj)); // [ 'a', 'b' ],替代 Object.getOwnPropertyNames
console.log(Reflect.deleteProperty(obj, 'b')); // true,替代 delete obj.b
console.log(obj); // { a: 1 }

// 用途三:调用函数 with this(替代 call 的现代写法)
const fn = function (x) { return this.base + x; };
console.log(Reflect.apply(fn, { base: 10 }, [5])); // 15

// 用途四:与 defineProperty 的差异(返回值 vs 抛错)
const ok = Reflect.defineProperty(obj, 'c', { value: 3, writable: false });
console.log(ok); // true
console.log(obj.c); // 3

// 一个完整的"读写校验 + 默认值"代理模板(生产常用):
function withDefaults(target, defaults) {
  return new Proxy(target, {
    get(obj, prop) {
      const v = Reflect.get(obj, prop);
      return v === undefined ? defaults[prop] : v; // 缺属性给默认值
    },
  });
}
const cfg = withDefaults({ port: 8080 }, { host: 'localhost', port: 3000 });
console.log(cfg.host, cfg.port); // localhost 8080(有值不覆盖)
```

★ 重点结论:Proxy + Reflect 是一对搭档

拦截器里的转发一律用 Reflect(保证与原生行为一致、返回值类型正确);业务代码里 Reflect 用来替代容易踩坑的老 API。读到框架源码中的 Reflect.get/set,就是在"转发原始操作",这是 Vue/ MobX 响应式、ORM 魔法的最底层。

§10.6 原型链操纵与动态元编程边界

```
// 动态原型操纵:运行时改方法(第6章讲过,这里从"元编程"视角再看)
function Greeter(name) {
  this.name = name;
}
Greeter.prototype.hello = function () { return `你好,${this.name}`; };

// 运行时给原型追加方法(影响所有实例)——这就是"猴子补丁"
Greeter.prototype.hello2 = function () { return `再次问好,${this.name}`; };
const g = new Greeter('李雷');
console.log(g.hello2()); // 再次问好,李雷

// 动态属性:Object.defineProperty 精细控制
const ticket = {};
Object.defineProperty(ticket, 'price', {
  value: 100,
  writable: false, // 只读
  enumerable: false, // 不出现在 for...in / Object.keys
  configurable: false, // 不可删除/重定义
});
// ticket.price = 200; // 严格模式 TypeError(只读)
console.log(Object.keys(ticket)); // [] ← 不可枚举
console.log(ticket.price); // 100

// 元编程边界:什么能做什么不能做
// 能:拦截读写、动态加方法、改原型、defineProperty、装饰(第12章)
// 不能:没有宏、没有编译期代码生成、不能拦截"私有字段 #"的访问
// # 私有字段天然绕过 Proxy(规范保证),所以"绝对私有"的数据用 #
```

⚠ 常见误区 10.3:猴子补丁原型是生产大忌

给 `Array.prototype` 等内建原型加方法(如 `Array.prototype.groupBy = ...`)会影响所有代码,库之间互相踩踏,还可能在引擎升级时与新 API 冲突。生产规则:应用代码可以小范围改自己的类原型;库代码禁止;内建原型禁止(用 `Array.prototype.groupBy.call(arr, fn)` 风格或包装函数替代)。

§10.7 编译期算力对照与生产判断

方案	机制	算力位置	代价
C++ 模板	编译期生成实例化代码	编译期	二进制膨胀、编译慢
Java 泛型	编译期检查,运行期擦除	编译期(检查)/运行期(擦除)	反射受限、基本类型要装箱
TypeScript	编译期检查,输出纯 JS	编译期(检查)	运行期无检查(靠类型声明保证)
JavaScript 动态	运行时动态分发	运行期	类型错误后置、无编译期保证
宏(#define / macro)	预处理文本替换	预处理	无类型、不可调试

🚀 工程建议 10.1:元编程何时该用,何时别用

该用:跨切面逻辑(日志、校验、权限、埋点)、数据绑定/响应式框架、ORM/序列化、测试替身(mock)。别用:业务代码里的"灵巧魔法"——Proxy 让调试断点看不到真实数据流,代码评审者要花三倍时间。判断标准:这个拦截是"对所有对象统一生效"还是"只对这一个对象",前者值得抽象成代理,后者直接写普通函数。框架内部多用,业务代码少用,这是 JS 元编程的生产共识。

本章速查表

主题	速查要点
C 的泛化方案	void* + 长度、宏、复制多份——各有痛点
JS 的"泛型"	动态类型天然接受一切类型;类型安全靠运行时校验
TS 泛型	function f<T>(x: T[]): T; T extends {length} 约束;编译期检查运行期擦除
Proxy 用途	拦截读写:日志、校验、默认值、隐藏字段、响应式
13 种拦截	get/set/has/deleteProperty/ownKeys/apply/construct 等
set 返回值	必须返回布尔(Reflect.set 转发)
Reflect 用途	拦截器内转发原始操作;替代 in/delete/apply 老写法
Reflect.ownKeys	返回自身全部键(含不可枚举与 Symbol)
动态原型	运行时加方法影响所有实例(猴子补丁);内建原型禁止
defineProperty	writable/enumerable/configurable 三开关精细控制
元编程边界	不能拦截 # 私有字段;没有宏/编译期代码生成
编译期算力	C++ 生成代码 > TS/Java 检查+擦除 > JS 运行时动态
生产判断	跨切面统一逻辑用代理;单点业务逻辑写普通函数

最小必会清单

- 能说出 C 的 void*/宏/复制多份三种泛化方案的痛点
- 能解释"JS 动态类型天然泛型"及代价(类型错误后置)
- 能看懂 TS 泛型 `function f<T>(x: T[]): T` 并说明运行期擦除
- 能写出 Proxy 的 get/set 拦截器(日志代理)
- 能说出 set 拦截为什么必须返回布尔
- 能用 Proxy 实现"只读对象"或"默认值对象"
- 能说出 Reflect 在拦截器里的作用(转发原始操作)
- 能用 defineProperty 实现只读不可枚举字段
- 能说出猴子补丁内建原型的生产禁令及替代
- 能说出 Proxy 不能拦截 # 私有字段

自测题

Q 1. C 的 `void*` 泛化与 JS 的动态类型,本质区别是什么?

✓ 参考答案

`void*` 需要显式强转与传长度,解引用错误是 UB;JS 直接接受任意值,少了解引用,但类型错误同样后置到运行期,且可能静默产出错误结果。

Q 2. 写出一个 Proxy,使 `obj.secret` 读取时返回 `'***'`,且 `'secret' in proxy` 为 `false`。

✓ 参考答案

get 拦截 `secret` 返回 `'***'`;has 拦截 `secret` 返回 `false`,其余 Reflect 转发。

Q 3. Proxy 的 set 拦截器为什么必须返回布尔?忘了 return 会怎样?

✓ 参考答案

规范要求 `set/deleteProperty` 返回布尔表示操作是否成功,严格模式下返回非布尔(undefined)会抛 `TypeError`。

Q 4. `Reflect.get(obj, prop)` 与 `obj[prop]` 的区别?拦截器里为什么用前者?

✓ 参考答案

两者结果等价,但 Reflect 以函数形式调用,可带 `receiver` 参数控制 `this`,且返回值/行为在拦截器语境下与规范一致,是转发原始操作的规范姿势。

Q 5. 为什么给 `Array.prototype` 加自定义方法被禁止?正确的替代是什么?

✓ 参考答案

会污染全局影响所有代码,库之间冲突,引擎升级可能撞名。替代:独立函数 `function groupBy(arr, keyFn) {...}` 或包装类。

Q 6. 说出 C++ 模板、Java 泛型、JS 动态三者的"算力位置"。

✓ 参考答案

C++ 编译期生成代码;Java 编译期检查 + 运行期擦除;JS 运行期动态分发。依次更灵活、更慢、更少编译期保证。

章末实验题:泛型容器与元编程工具箱

实验 10:泛型容器与元编程工具箱(meta-toolkit)

实验目标:实现类型受约束的栈容器 + Proxy 元编程工具箱(日志/校验/深代理/默认值),全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:动态类型"泛型"、运行时校验):TypedStack 容器,构造参数是校验函数,入栈校验、非法抛 TypeError
- 任务 2(覆盖:Map/Set 内置泛型结构):用 Map 实现"按 id 索引"的缓存,Set 做去重统计
- 任务 3(覆盖:Proxy get/set、Reflect 转发):日志代理记录访问与修改次数,导出统计
- 任务 4(覆盖:校验代理、deleteProperty):账户对象代理,余额非负校验、关键字段禁删
- 任务 5(覆盖:默认值代理):withDefaults 代理,缺属性返回默认值
- 任务 6(覆盖:深代理):递归代理嵌套对象,嵌套属性同样拦截(用 typeof value === 'object' 递归)
- 任务 7(覆盖:defineProperty、原型动态):defineProperty 创建只读字段;运行时给原型加方法并验证所有实例可见
- 任务 8(覆盖:TS 方向):输出一段"TS 泛型对照"注释,说明 Stack<T> 声明如何给任务 1 补静态类型

实验环境:Node.js 20+,运行 `node metatoolkit.js`。

第一步 **写容器:**TypedStack 与 Map 索引缓存,验证"类型约束"与"任意键"

第二步 **写代理工具箱:**makeLogger/makeValidator/withDefaults/deepProxy 四个工厂函数

第三步 **后写演示:**每个工具一个迷你场景,输出拦截日志与统计

✅ 参考答案要点

核心: TypedStack + 四类代理工厂 + defineProperty/原型演示。约 140 行,覆盖全章知识点。

```
// metatoolkit.js — Node.js 20+ 运行:node metatoolkit.js
'use strict';

// 1. 类型受约束的栈(覆盖:动态类型"泛型"、运行时校验)
function createTypedStack(name, validate) {
  const items = [];
  return {
    push(item) {
      if (!validate(item)) {
        throw new TypeError(`${name} 拒绝元素:${item}`);
      }
      items.push(item);
    },
    pop() { return items.pop(); },
    get size() { return items.length; },
  };
}
const numStack = createTypedStack('数字栈', (v) => typeof v === 'number');

// 2. Map/Set 内置结构(覆盖:任意键、去重)
function indexDemo() {
  const byId = new Map();
  byId.set(1001, { name: '李雷' });
  byId.set('k-2', { name: '韩梅梅' }); // 字符串键
  console.log('Map 索引:', byId.get(1001).name, byId.get('k-2').name);

  const seen = new Set(['a', 'b', 'a', 'c', 'b']);
  console.log('Set 去重后数量:', seen.size); // 3
}

// 3. 日志代理(覆盖:Proxy get/set、Reflect 转发、闭包统计)
function makeLogger(target) {
  const stats = { get: 0, set: 0 };
  const proxy = new Proxy(target, {
    get(obj, prop) {
      stats.get++;
      return Reflect.get(obj, prop);
    },
    set(obj, prop, value) {
      stats.set++;
      return Reflect.set(obj, prop, value);
    },
  });
  proxy.__stats = stats; // 附加统计出口
  return proxy;
}

// 4. 校验代理(覆盖:set/deleteProperty 拦截)
function makeValidator(target, rules) {
  return new Proxy(target, {
    set(obj, prop, value) {
      const rule = rules[prop];
      if (rule && !rule(value)) {
        throw new TypeError(`字段 ${prop} 校验失败:${value}`);
      }
    }
  });
}
```

```

        return Reflect.set(obj, prop, value);
    },
    deleteProperty(obj, prop) {
        if (rules[prop]?.locked) return false; // 锁定字段禁删
        return Reflect.deleteProperty(obj, prop);
    },
});
}

// 5. 默认值代理(覆盖:get 拦截、默认值)
function withDefaults(target, defaults) {
    return new Proxy(target, {
        get(obj, prop) {
            const v = Reflect.get(obj, prop);
            return v === undefined ? defaults[prop] : v;
        },
    });
}

// 6. 深代理(覆盖:递归拦截、嵌套对象)
function deepProxy(target, handler) {
    const wrap = (obj) => {
        if (obj !== null && typeof obj === 'object') {
            return new Proxy(obj, {
                ...handler,
                get(t, p) {
                    const v = Reflect.get(t, p);
                    return wrap(v); // 值也是对象就继续包
                },
                set(t, p, val) {
                    if (typeof handler.set === 'function') handler.set(t, p, val);
                    return Reflect.set(t, p, val);
                },
            });
        }
        return obj;
    };
    return wrap(target);
}

// 7. defineProperty 与动态原型(覆盖:属性开关、运行时加方法)
function propertyDemo() {
    const ticket = {};
    Object.defineProperty(ticket, 'price', {
        value: 100, writable: false, enumerable: false, configurable: false,
    });
    console.log('只读且不可枚举:', ticket.price, Object.keys(ticket).length === 0);

    function Greeter(name) { this.name = name; }
    Greeter.prototype.hello = function () { return `你好,${this.name}`; };
    Greeter.prototype.bye = function () { return `${this.name} 再见`; }; // 运行时追加
    console.log(new Greeter('李雷').bye()); // 所有实例立即可用
}

// 8. TS 对照注释(覆盖:TS 泛型方向)

```

```

/*
 * TypeScript 对照(需 .ts):
 * function first(arr: T[]): T | undefined { return arr[0]; }
 * class Stack { private items: T[] = [];
 *   push(item: T): void { this.items.push(item); }
 *   pop(): T | undefined { return this.items.pop(); } }
 * 上面的 numStack 若用 TS 写,校验函数可换成类型参数 T 的静态约束,
 * 运行时校验(assert)与编译期检查(T)互补使用。
 */

// 9. 主流程
function main() {
  indexDemo();

  // 日志代理
  const user = makeLogger({ name: '李雷' });
  user.name = '韩梅梅';
  console.log('日志统计:', user.__stats);    // { get: 0, set: 1 }
  console.log(user.name);                    // 韩梅梅(get 计数 +1)

  // 校验代理
  const acc = makeValidator({ balance: 0 }, {
    balance: (v) => typeof v === 'number' && v >= 0,
  });
  acc.balance = 50;
  // acc.balance = -1;    // ✖ TypeError:校验失败
  // delete acc.balance; // 若规则加 locked:true 则被拒绝

  // 默认值代理
  const cfg = withDefaults({ port: 8080 }, { host: 'localhost', port: 3000 });
  console.log('默认值代理:', cfg.host, cfg.port); // localhost 8080

  // 深代理:嵌套修改也拦截
  const nested = deepProxy({ a: { b: { c: 1 } } }, {
    set(t, p, v) { console.log(`深代理拦截:${String(p)} = ${v}`); },
  });
  nested.a.b.c = 99;                // 触发深代理拦截
  console.log('深代理读取:', nested.a.b.c);

  // TypedStack
  numStack.push(1);
  numStack.push(2);
  console.log('数字栈弹出:', numStack.pop(), '| 栈大小:', numStack.size);

  propertyDemo();
  console.log('实验完成(TS 对照见源码注释)');
}

main();

```

设计说明:TypedStack 用"校验函数注入"模拟泛型约束,呼应 TS 的 `Stack<T>`;代理工具箱四个工厂覆盖 get/set/deleteProperty 三类拦截与深递归;日志代理的统计用闭包附加在 proxy 上;深代理递归包裹嵌套值,直观

展示"响应式框架的雏形";defineProperty 演示只读+不可枚举,动态原型演示运行时加方法。TS 对照注释给出任务 1 的静态版本,为第14章铺路。

下一章预告:第11章 反射与动态特性 —— typeof/instanceof 的坑、Object 反射全家、属性描述符,以及手写 JSON 序列化器。